Phase 0: Project Planning & Environment Setup (Day 1 to 2)

0.1 Project Structure:

```
financial-llm-prompting/
│
├── config/
│   ├── settings.py
│   ├── prompts.yaml
│   └── constants.py
│
├── data/
│   ├── raw_statements/
│   ├── processed/
│   ├── eval_dataset/
│   ├── expected_outputs/
│   └── clean_text_backup/
│
├── src/
│   ├── extraction/
│   ├── preprocessing/
│   ├── prompts/
│   ├── llm/
│   ├── engine/
│   ├── evaluation/
│   ├── visualization/
│   └── utils/
│
├── outputs/
│   ├── responses/
│   ├── experiments/
│   ├── reports/
│   ├── charts/
│   └── logs/
│
├── notebooks/
├── app.py
├── requirements.txt
├── README.md
├── .env
└── .env.example
```

0.2 Environment

Install

- Python 3.11
- Streamlit
- Google Gemini API SDK
- pdfplumber
- PyMuPDF
- pandas
- matplotlib
- python-dotenv
- openpyxl
- Jupyter Notebook

0.3 Configuration

Create

```
config/settings.py
```

Store

- Gemini Model
- Temperature
- Max Output Tokens
- Timeout
- Retry Count

Never hardcode values inside the code.

0.4 API Security

Store Gemini API key inside

```
.env
```

Load using

```
python-dotenv
```

Include

```
.env.example
```

inside GitHub repository.

0.5 Backup Environment

Prepare

- Google Colab notebook
- Local development environment

This ensures experiments can continue even if local hardware has limitations.

Phase 1: Dataset Collection & Ground Truth Creation (Day 2 to 5)

This phase is the foundation of the research.

Collect

10–20 financial statements including

- Balance Sheet
- Income Statement

Sources may include

- Public company reports
- Accounting education datasets
- SEC sample filings
- University accounting resources

For every statement create a Ground Truth file.

Example

```
statement_01.json
```

```
{
    pdf_path,

    expected_summary,

    expected_facts,

    expected_ratios,

    expected_risks,

    expected_recommendations
}
```

Store

```
data/eval_dataset/
```

Also prepare

```
clean text versions

CSV versions
```

for fallback if PDF extraction fails.

Phase 2: PDF Extraction & Data Preprocessing (Day 5 to 8)

Build

```
src/extraction/pdf_extractor.py
```

Primary extractor

```
pdfplumber
```

Fallback

```
PyMuPDF
```

Responsibilities

- Extract text
- Extract tables
- Preserve financial values
- Remove headers
- Remove footers
- Normalize whitespace
- Preserve negative numbers
- Preserve currency formatting

Output

```
Processed JSON

Processed Text
```

Example

```
{
    Revenue: ...

    Expenses: ...

    Assets: ...

    Liabilities: ...
}
```

Save processed files inside

```
data/processed/
```

Phase 3: Prompt Engineering Module (Day 8 to 11)

Build

```
src/prompts/
```

Folder

```
builder.py

templates/

basic_v1.txt

role_v1.txt

structured_v1.txt

fewshot_v1.txt
```

Prompt Builder dynamically injects the extracted financial statement into the selected template.

Prompt Strategies

Basic Prompt

Simple direct analysis.

Role-Based Prompt

LLM acts as

- CPA
- CFA
- Senior Financial Analyst

Structured Prompt

Force the model to generate sections including

- Executive Summary
- Revenue
- Expenses
- Profitability
- Liquidity
- Solvency
- Financial Ratios
- Risks
- Recommendations

Few-Shot Prompt

Provide

Example Financial Statement

↓

Ideal Analysis

↓

Target Statement

↓

Generate Similar Analysis

Prompt Versioning

Instead of

```
structured.txt
```

Use

```
structured_v1

structured_v2

structured_v3
```

This improves experiment reproducibility.

Phase 4: LLM Analysis Engine (Day 11to 14)

Create

```
src/llm/gemini_client.py
```

Responsibilities

- Gemini API connection
- Retry logic
- Timeout handling
- Token logging
- Error handling

Build

```
src/engine/analyzer.py
```

Pipeline

```
Financial Statement

↓

Prompt Builder

↓

Gemini API

↓

Analysis

↓

Store Results
```

Optional

Create adapter interface for

- GPT-4.1 Mini
- Llama 3.1 8B

Enable via configuration flag without changing the main pipeline.

Phase 5: Response Processing & Storage (Day 14 to 16)

Every experiment should be saved.

Instead of storing only text,

store complete metadata.

```
{
    statement_id,
    strategy,
    prompt_version,
    model,
    extracted_text,
    prompt,
    response,
    structured_json,
    latency,
    timestamp,
    input_tokens,
    output_tokens
}
```

Save inside

```
outputs/responses/
```

Structured JSON Output

Ask Gemini to generate

1. Human-readable report
2. Structured JSON

Example

```json
{
    "Revenue": "...",
    "Expenses": "...",
    "Net Profit": "...",
    "Current Ratio": "...",
    "Debt to Equity": "...",
    "ROE": "...",
    "Risks": "...",
    "Recommendations": "..."
}
```

This makes evaluation significantly easier.

Phase 6: Evaluation Framework (Day 16 to 20)

This is the core research contribution.

Build

```
src/evaluation/evaluator.py
```

Run

```
20 Statements

×

4 Prompt Strategies

=

80 Experiments
```

Evaluation Metrics

1. Financial Fact Extraction Accuracy

Compare

Revenue

Assets

Liabilities

Profit

Ratios

against Ground Truth.

2. Completeness

Checklist scoring.

Did the model discuss

- Revenue
- Expenses
- Profit
- Liquidity
- Solvency
- Risks
- Recommendations

Score

```
Covered Topics

/

Expected Topics
```

3. Hallucination Rate

Identify

- Numbers not present
- Ratios not supported
- False claims

Calculate

```
Hallucinated Claims

/

Total Claims
```

4. Response Time

Automatically measured.

5. User Rating

Collect

1–5

ratings inside Streamlit.

6. Optional Advanced Metrics

- Precision
- Recall
- F1 Score

for extracted financial facts.

Save

```
results.csv

experiment_summary.csv
```

Phase 7: Visualization & Analytics (Day 20 to 22)

Build

```
src/visualization/
```

Generate

- Accuracy comparison
- Completeness comparison
- Hallucination comparison
- Response time comparison
- User rating comparison

Charts

- Bar Chart
- Line Chart
- Box Plot (optional)

Export

PNG

PDF

CSV

Phase 8: Streamlit Web Application (Day 22 to 25)

Main Features

Upload PDF

↓

Extract Financial Statement

↓

Select Prompt Strategy

or

Run All Strategies

↓

Generate Analysis

↓

Compare Results

↓

Show Evaluation Metrics

↓

Download Report

User Interface

Sidebar

- Upload PDF
- Prompt Strategy
- Model Selection
- Run Button

Main Page

- PDF Preview
- Extracted Text
- Generated Analysis
- Structured JSON View
- Comparison Table
- Charts
- Rating Widget

Phase 9: Batch Experiments & Error Analysis (Day 25 to 27)

Run

```
10-20 Statements

×

4 Strategies

=

40-80 Experiments
```

using Jupyter Notebook.

Aggregate

- Mean Accuracy
- Mean Completeness
- Mean Hallucination
- Mean Response Time

Perform qualitative error analysis.

Examples

- Incorrect ratios
- Missing sections
- Hallucinated values
- Extraction failures

Document why they occurred.

Phase 10: Documentation & Report Writing (Day 27 to 29)

Prepare

Literature Review

Include

- BloombergGPT
- FinGPT
- Prompt Engineering Guide
- Additional Financial NLP papers (optional)

Methodology

- Dataset
- PDF Extraction

- Prompt Design
- LLM Configuration
- Evaluation Metrics

Experimental Setup

- Hardware
- Software
- Gemini Configuration
- Temperature
- Dataset Size

Results

Include

- Tables
- Charts
- Metric Comparisons
- Error Analysis

Limitations

- PDF extraction quality
- Small evaluation dataset
- Single-model dependence
- Subjective hallucination labeling

Future Work

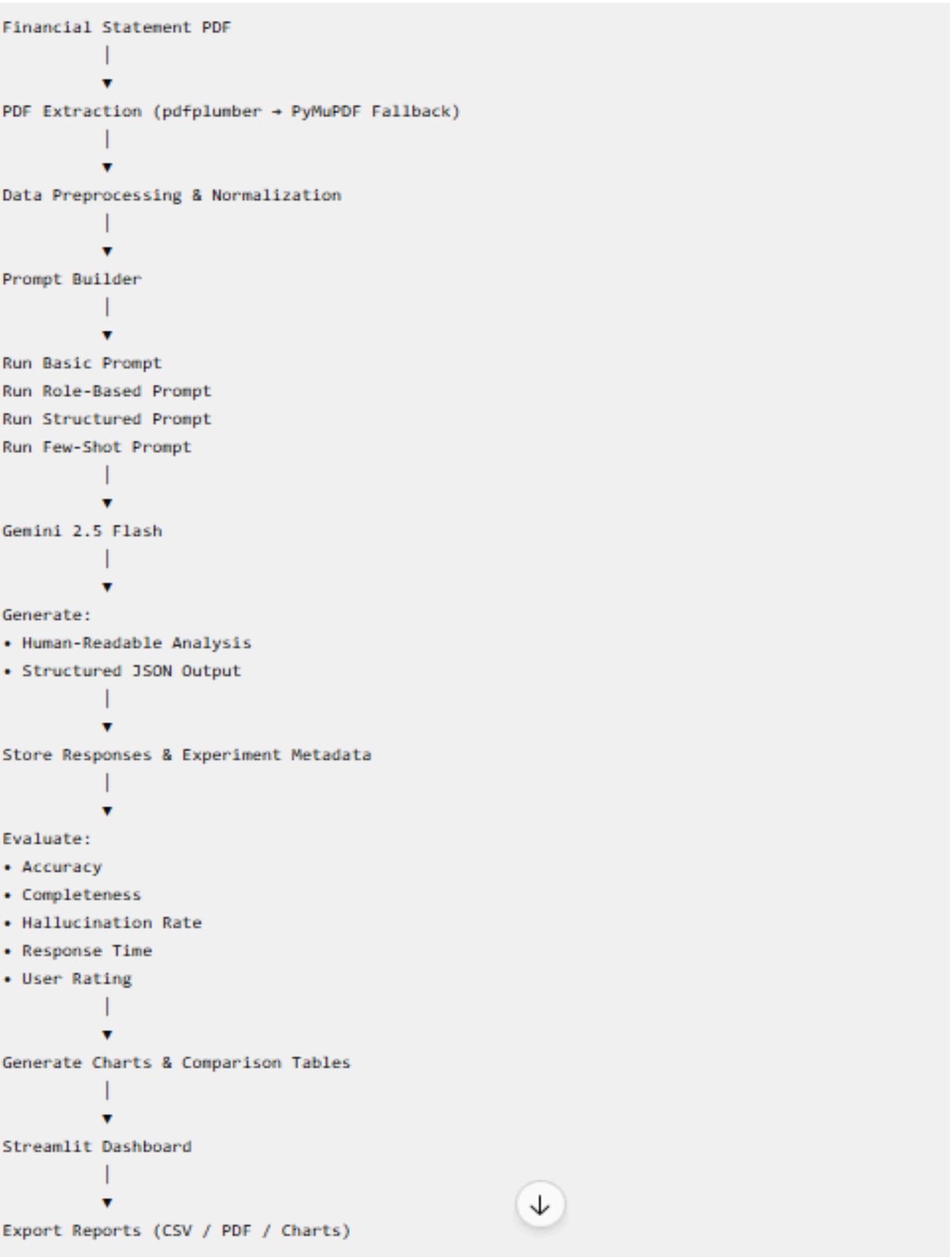
- Larger datasets
- Multi-model comparison
- RAG integration
- Fine-tuning
- OCR for scanned PDFs

- Automated financial ratio validation

Phase 11: Final Testing & Delivery (Day 29 to 30)

- Refactor code
- Add comments and docstrings
- Finalize `README.md`
- Include setup instructions
- Verify `.env.example`
- Test on unseen financial statements
- Record a short demo video (optional but recommended)
- Validate results against the project's target metrics before submission

Final End-to-End Workflow


Financial Statement PDF
PDF Extraction (pdfplumber → PyMuPDF Fallback)
Data Preprocessing & Normalization
Prompt Builder
Run Basic Prompt
Run Role-Based Prompt
Run Structured Prompt
Run Few-Shot Prompt
Gemini 2.5 Flash
Generate:
• Human-Readable Analysis
• Structured JSON Output
Store Responses & Experiment Metadata
Evaluate:
• Accuracy
• Completeness
• Hallucination Rate
• Response Time
• User Rating
Generate Charts & Comparison Tables
Streamlit Dashboard
Export Reports (CSV / PDF / Charts)